PE-JV-003

Aula 05 - Containers, CI/CD & Deploy sem Downtime

Aquecimento

Kahoot

Revisão da Aula 04: 2PC vs Saga, Coreografia vs Orquestração, Compensação, Transactional Outbox e CQRS.

Aguarde as orientações do professor.

O Desafio de Engenharia

Na última aula, coordenamos transações distribuídas entre Checkout, Pagamento e Estoque. Agora precisamos publicar uma atualização urgente no **Checkout** em plena Black Friday, com **5.000 TPS** ativos.

O incidente na Black Friday:
O pipeline de produção recompilou a branch `main`, sobrescreveu a imagem `checkout:latest` e iniciou a atualização. No boot da primeira instância nova, o **Flyway** renomeou a coluna `status_pagamento` para `estado_saga`.

O problema:
Os 80% de servidores que ainda rodavam a versão antiga quebraram na hora (`column does not exist`). Ao tentar voltar a tag `:latest`, ninguém sabia qual binário rodava antes e a versão antiga continuou falhando contra o banco alterado. Como entregar e reverter versões sem derrubar pedidos em andamento?

A Missão do Dia

No seu documento de System Design (Aula 05):

- Definir o padrão de empacotamento e redução de superfície de ataque da imagem de produção.
- Estruturar a esteira de confiança com promoção de artefato imutável e gates automatizados de segurança.
- Compreender o papel de Infraestrutura como Código (IaC), orquestração e parametrização entre ambientes.
- Desenhar a estratégia de rollout progressivo combinada com migração de banco de dados compatível com rollback imediato.

Sala de Aula Invertida: A Pesquisa

Acesse os materiais compartilhados e pesquise:

1. **Construção vs Execução:** Por que ferramentas de build e shell não devem ir para produção? Como auditar vulnerabilidades em todas as camadas da imagem?
2. **Build, Release, Run:** Por que recompilar o código em cada ambiente quebra a rastreabilidade e por que tags como `:latest` são mutáveis?
3. **Infraestrutura como Código & Orquestração:** Como provisionar infraestrutura sem ações manuais (*ClickOps*) e injetar configurações por ambiente na mesma imagem?
4. **Migrações & Rollout sem Downtime:** Como evoluir colunas no PostgreSQL enquanto duas versões da aplicação coexistem, e quais os trade-offs entre **Canary Release** e **Blue-Green Deployment**?

O Desafio Arquitetural

No seu documento de System Design, fundamente suas decisões técnicas:

1. **Empacotamento, Superfície de Ataque e Promoção Imutável:**
   - Como empacotar o serviço para produção reduzindo a superfície de ataque (o que entra e o que fica de fora da imagem final)?
   - Como garantir que o binário em produção seja idêntico ao homologado (parametrizando variáveis por ambiente sem recompilar) e impedir que tags sobrescritas inviabilizem o rollback?
2. **Estratégia de Rollout e Migração de Banco sem Downtime:**
   - Qual estratégia de deploy (**Canary Release** ou **Blue-Green**) guiará a atualização do Checkout e quais os trade-offs da escolha?
   - Qual o passo a passo para evoluir a coluna `status_pagamento` para `estado_saga` sem quebrar as instâncias da versão anterior nem impedir o rollback imediato?
3. **Guardrails de Segurança, Governança & Validação da Release:**
   - Quais gates automatizados e *guardrails* de governança devem existir do commit ao deploy para proteger a entrega sem perder rastreabilidade?
   - Como testar e comprovar automaticamente na pipeline que a imagem final construída funciona antes de receber tráfego de clientes?

Debate Técnico

É hora de debater com o comitê de engenharia (Staff Engineer):

- **Unidade de Entrega:** Por que o padrão OCI substituiu o envio de `.jar` isolado, mesmo quando não usamos Docker em produção?
- **Superfície de Ataque vs Diagnóstico:** O que ganhamos com imagens *Distroless* e como investigamos problemas em produção sem terminal (`sh`)?
- **Identidade e Guardrails na Esteira:** Tag vs Digest (`sha256`), verificações de segurança em camadas, SBOM e governança do commit ao deploy.
- **Quem Faz o Quê na Infra:** A fronteira conceitual entre Terraform, Kubernetes e Helm Charts.
- **Coexistência no Banco:** Por que migrações destrutivas quebram deploys graduais e como o *Expand-Contract* garante rollback seguro.

Projeto Individual (System Design)

Cenário de Engenharia (Release do Checkout na Black Friday):

- **Operação Crítica:** Serviço de Checkout processando **5.000 TPS** com múltiplas réplicas ativas conectadas ao mesmo PostgreSQL.
- **Mudança de Schema:** Necessidade de evoluir a coluna `status_pagamento` para `estado_saga` sem interromper compras em andamento.
- **Requisito de Governança:** Capacidade de auditar exatamente qual binário está em execução e reverter qualquer deploy em segundos.

Sua Missão:

- Responda às decisões do **Desafio Arquitetural** aplicadas diretamente ao cenário acima.
- Utilize o **Template da Aula 05** (disponível no LMS) e versione a entrega no repositório do seu projeto com commits atômicos e descritivos.

Laboratórios Práticos (Opcional)

Disponibilizamos no LMS o guia de laboratórios e ferramentas oficiais da indústria:

- **Docker Multi-Stage & Distroless:** Construção prática comparando tamanho, camadas e superfície de ataque entre imagens JDK padrão e JRE Distroless.
- **Trivy Security Scanner:** Execução de auditoria dupla separando vulnerabilidades do `pom.xml` (SCA) e vulnerabilidades do sistema operacional da imagem.
- **Flyway & Expand-Contract:** Simulação de evolução de schema em etapas garantindo compatibilidade simultânea entre duas versões da API.

Próxima Aula: Observabilidade & Diagnóstico em Produção

Colocamos uma imagem *Distroless* (sem shell!) rodando em *Canary Release*. Como sabemos se os 5% de tráfego estão saudáveis e como investigamos quando a latência dispara?

Tópicos Conceituais:

- **Os Três Pilares e Correlação:** Logs estruturados (JSON), métricas de cardinalidade controlada e tracing distribuído com OpenTelemetry.
- **Saturação e Gargalos Silenciosos:** Como detectar exaustão de pools e travas antes que virem erros 5xx.
- **Diagnóstico Sistemático (RCA):** Roteiro guiado por dados para investigar degradação de latência p99 entre múltiplos microsserviços.

Referências para Leitura Prévia:

- 📄 OpenTelemetry: What is OpenTelemetry?
- 📄 Grafana Labs: The RED Method
- 📄 Brendan Gregg: The USE Method

Referências da Aula 05 (Containers & CI/CD)

Links oficiais para estudo e aprofundamento:

- 📄 **Multi-Stage & Distroless:** Docker Multi-stage builds e Google Distroless Images
- 📄 **Build, Release, Run:** The Twelve-Factor App: Build, release, run
- 📄 **Scanner de Vulnerabilidades:** Trivy Documentation
- 📄 **Infraestrutura como Código:** Red Hat: O que é IaC? e HashiCorp Terraform Intro
- 📄 **Orquestração & Charts:** Red Hat: O que é Kubernetes? e Helm Quickstart
- 📄 **Migrações & Expand-Contract:** Martin Fowler: Parallel Change e Redgate Flyway Migrations
- 📄 **Estratégias de Deploy:** Canary Release e Blue-Green Deployment (Martin Fowler)

Obrig.ADA